Compte rendu – TP1 : Création de service web SOAP

Module : Développement Orienté Service – 3IDL, Département IDL, Institut Supérieur d'Informatique

Année universitaire : 2026/2027

Réalisé par : Hedi Cherni, Hadir Ben Khalfallah & Mariem Bouchendira

Date : 10 novembre 2026

1. Introduction

Ce TP a pour but d'initier à la création d'un service web SOAP en Java avec Spring Boot et Spring Web Services (Spring-WS). Le service est développé selon l'approche **contrat d'abord** (*contract-first*) : on définit d'abord le contrat (schéma XSD) à partir duquel sont générées les classes Java et le WSDL, puis on implémente la logique métier.

Le service exposé permet de récupérer les informations d'un étudiant (nom, niveau, adresse) à partir de son nom.

2. Objectifs

- Créer un projet Spring Boot avec les dépendances nécessaires aux services web SOAP.
- Définir le contrat du service dans un fichier XSD et générer les classes Java avec JAXB.
- Implémenter un endpoint SOAP et un référentiel de données.
- Configurer la publication du WSDL.
- Déployer le service et le tester avec un client SOAP.

3. Environnement de travail

Élément	Détail
Framework	Spring Boot, Spring Web Services
Langage	Java (le code utilise `jakarta.annotation`, donc Spring Boot 3 / Java 17 ou supérieur)
Outil de build	Maven (`mvn clean install`)
Génération de code	jaxb2-maven-plugin (XJC)
Dépendance WSDL	wsdl4j
Serveur	Tomcat embarqué, port 8080
Outil de test	Client HTTP envoyant des requêtes SOAP (Postman / SoapUI)

4. Architecture du service

Le projet est organisé autour des éléments suivants :

Élément	Rôle
`student.xsd`	Contrat : définit StudentDetailsRequest, StudentDetailsResponse et le type Student

Élément	Rôle
Classes générées (`org.exemple.student`)	StudentDetailsRequest, StudentDetailsResponse, Student, générées par XJC
`StudentRepository`	Source de données en mémoire (HashMap) avec la méthode findStudent()
`StudentEndpoint`	Reçoit les requêtes SOAP et délègue la recherche au référentiel
`config`	Configure la servlet SOAP et expose le WSDL
`DemoApplication`	Point d'entrée Spring Boot

Flux d'une requête : client SOAP → **MessageDispatcherServlet** (/service/*) → **StudentEndpoint** (sélection par namespace et localPart) → **StudentRepository** → **StudentDetailsResponse**.

5. Étapes de réalisation

5.1 Création du projet Spring Boot

Le projet est généré depuis Spring Initializr (Maven, Java) avec la dépendance **Spring Web Services**, puis importé dans l'IDE comme projet Maven.

5.2 Ajout de la dépendance wsdl4j

Dans le fichier `pom.xml` :

```
<dependency>
  <groupId>wsdl4j</groupId>
  <artifactId>wsdl4j</artifactId>
</dependency>
```

5.3 Définition du contrat : student.xsd

Le schéma, placé dans `src/main/resources`, utilise le namespace `http://www.exemple.org/student` avec elementFormDefault="qualified" :

- **StudentDetailsRequest** : contient un élément name (xs:string) ;
- **StudentDetailsResponse** : contient un élément Student de type tns:Student ;
- **Student** : type complexe composé de name (xs:string), standard (xs:int) et address (xs:string).

5.4 Génération des classes Java

Le plugin **jaxb2-maven-plugin** (version 1.6, goal xjc) est ajouté au pom.xml. Il lit les schémas dans src/main/resources et génère les classes dans src/main/java (clearOutputDir = false). XJC compile le XSD en classes Java annotées JAXB.

5.5 Création du référentiel de données : StudentRepository

Classe annotée @Component. Les données sont codées en dur dans une HashMap<String, Student> initialisée dans une méthode @PostConstruct :

Nom	Standard	Adresse
Sajal	5	Pune
Kajal	5	Chicago
Lokesh	6	Delhi
Sukesh	7	Noida

La méthode findStudent(String name) vérifie que le nom n'est pas nul (Assert.notNull) puis retourne students.get(name), c'est-à-dire **null** si le nom est inconnu.

5.6 Création de l'endpoint SOAP : StudentEndpoint

```
@Endpoint
public class StudentEndpoint {
    private static final String NAMESPACE_URI = "http://www.exemple.org/student";
    private final StudentRepository studentRepository;

    @Autowired
    public StudentEndpoint(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }

    @PayloadRoot(namespace = NAMESPACE_URI, localPart = "StudentDetailsRequest")
    @ResponsePayload
    public StudentDetailsResponse getStudent(@RequestPayload StudentDetailsRequest request) {
        Student student = studentRepository.findStudent(request.getName());
        StudentDetailsResponse response = new StudentDetailsResponse();
        if (student != null) {
            response.setStudent(student);
        }
        return response;
    }
}
```

Annotations utilisées :

- **@Endpoint** : enregistre la classe auprès de Spring-WS comme candidate au traitement des messages SOAP entrants ;
- **@PayloadRoot** : choisit la méthode de traitement selon le namespace et le localPart ;
- **@RequestPayload** : mappe le message entrant au paramètre de la méthode ;
- **@ResponsePayload** : mappe la valeur retournée à la réponse SOAP.

L'endpoint injecte le référentiel par le constructeur (au lieu d'un appel statique) et ne renseigne l'étudiant dans la réponse que s'il existe.

5.7 Configuration du service SOAP : classe config

Classe annotée @EnableWs et @Configuration, qui déclare trois beans :

- **MessageDispatcherServlet** : gère les requêtes SOAP. Il reçoit l'ApplicationContext pour reconnaître les autres beans, active setTransformWsdlLocations(true) et est enregistré sur le mapping /service/* ;
- **DefaultWsdl11Definition** (bean studentDetailsWsdl) : expose un WSDL 1.1 à partir du schéma, avec le port StudentDetailsPort, l'URI /service/student-details et le namespace http://www.exemple.org/student ;
- **XsdSchema** (SimpleXsdSchema) : charge student.xsd depuis le classpath.

Le WSDL est accessible à l'adresse `http://localhost:8080/service/studentDetailsWsdl.wsdl`.

5.8 Construction et démarrage

```
mvn clean install
java -jar target/<nom-du-projet>-0.0.1-SNAPSHOT.jar
```

Cette commande démarre Tomcat sur le port 8080 et déploie l'application.

6. Tests

Le service a été testé depuis l'IDE avec un fichier de requêtes HTTP (**rest-api_1.http**) envoyant des messages SOAP à l'adresse `http://localhost:8080/service`. Deux cas ont été vérifiés : un nom inconnu et un nom existant.

6.1 Test 1 : nom inexistant (« Sajals »)

Requête envoyée :

```
POST http://localhost:8080/service
Content-Type: text/xml; charset=UTF-8
SOAPAction: ""

<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
                  xmlns:stu="http://www.exemple.org/student">
    <soapenv:Header/>
    <soapenv:Body>
        <stu:StudentDetailsRequest>
            <stu:name>Sajals</stu:name>
        </stu:StudentDetailsRequest>
    </soapenv:Body>
</soapenv:Envelope>
```

Réponse obtenue (HTTP 200, 219 octets) :

```
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header/>
    <SOAP-ENV:Body>
        <ns2:StudentDetailsResponse xmlns:ns2="http://www.exemple.org/student"/>
    </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```



Figure 1 – Réponse du service pour un nom inexistant : StudentDetailsResponse vide

Analyse : la requête est reçue par la servlet et routée vers getStudent() grâce au namespace et au localPart StudentDetailsRequest ; une réponse SOAP valide est renvoyée (HTTP 200). Le nom « Sajals » n'existe pas dans le référentiel (qui contient « Sajal »), donc findStudent() retourne null et l'endpoint renvoie un StudentDetailsResponse **vide**, sans erreur.

6.2 Test 2 : nom existant (« Sajal »)

Seul le contenu de l'élément name change :

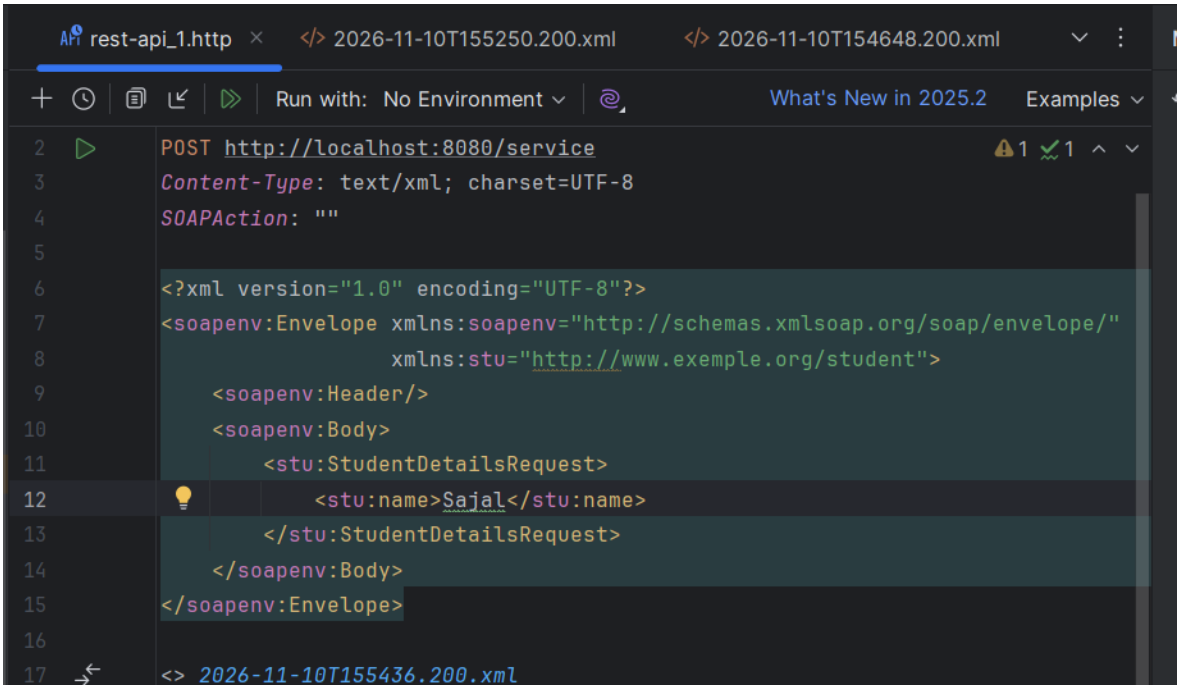


Figure 2 – Requête SOAP envoyée avec le nom « Sajal » (fichier rest-api_1.http)

Réponse obtenue (HTTP 200) :

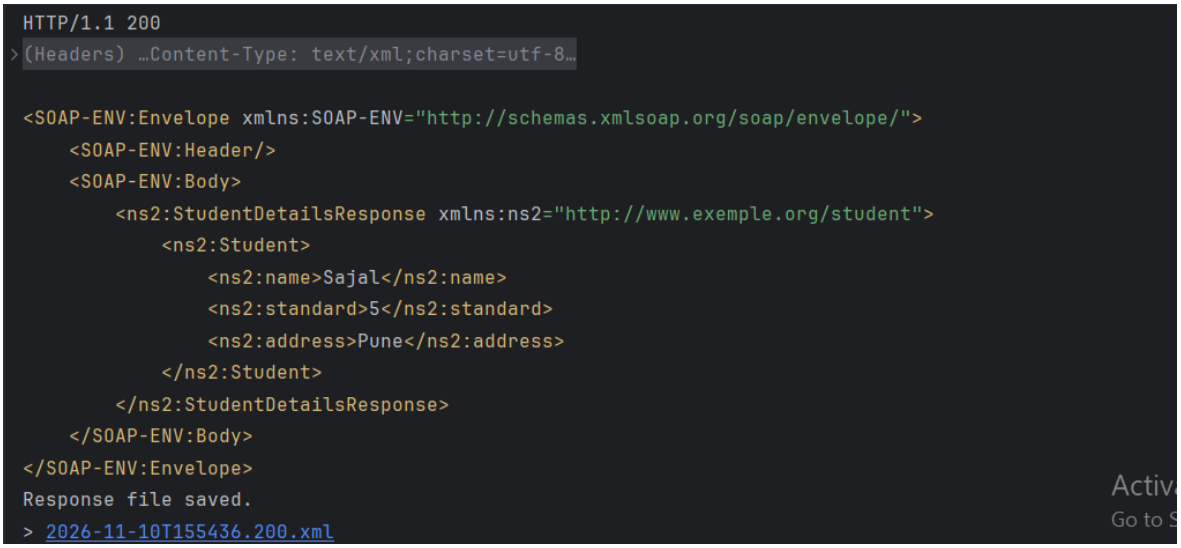


Figure 3 – Réponse du service : l'étudiant Sajal (standard 5, adresse Pune)

```
<ns2:StudentDetailsResponse xmlns:ns2="http://www.exemple.org/student">
    <ns2:Student>
        <ns2:name>Sajal</ns2:name>
        <ns2:standard>5</ns2:standard>
        <ns2:address>Pune</ns2:address>
    </ns2:Student>
</ns2:StudentDetailsResponse>
```

Analyse : cette fois, le référentiel trouve l'étudiant et la réponse contient l'élément Student avec ses trois champs (name, standard, address), conformément au contrat défini dans student.xsd. Le service fonctionne donc correctement dans les deux cas.

7. Difficultés et remarques

- Le sujet mentionne le JDK 7 ou 8, mais le code utilise jakarta.annotation.PostConstruct, qui nécessite Spring Boot 3 et Java 17 ou supérieur ; la configuration du projet doit être cohérente avec cela.
- Le sujet fait étendre WsConfigurerAdapter à la classe de configuration ; cette classe a disparu des versions récentes de Spring-WS, où @EnableWs suffit, comme dans le code réalisé.
- La recherche est sensible à la casse et à l'orthographe exacte du nom (Sajals ≠ Sajal : une simple faute de frappe a d'abord donné une réponse vide). Un nom inconnu produit une réponse vide plutôt qu'un message d'erreur.
- Amélioration possible : retourner un **SOAP Fault** ou un message explicite lorsque l'étudiant n'existe pas.
- Le point de terminaison de la requête de test est /service, alors que le WSDL déclare /service/student-details ; le service a répondu, mais l'URL du WSDL est celle à utiliser dans un client.

8. Conclusion

Ce TP a permis de mettre en place un service web SOAP complet avec Spring Boot en suivant l'approche contrat d'abord : définition du XSD, génération des classes JAXB, implémentation de l'endpoint et du référentiel, configuration et publication du WSDL. Les deux tests réalisés ont confirmé que le service est opérationnel (HTTP 200) : une réponse vide pour un étudiant inconnu et l'étudiant « Sajal » (standard 5, Pune) pour un nom existant, ce qui valide le routage par namespace et localPart ainsi que la conformité au contrat XSD. La seconde partie du TP, la création d'un client web service consommant ce service, constitue la suite du travail.